

Search Trees

1.The Dictionary ADT

a dictionary (also symbol-table,relation,map) ADT is used to represent a set of elements with key values.

each element has a key field and a data field

- Operations:
- Search(S,k):find an element in S with key value k .
- Insert(S,x):add x to S .
- Remove(S,x):remove element x from S .
- Remove(S,k):remove element with key value k from S .
- Min(S) and Max(S):find the element in S with minimum / maximum key.
- Successor(S,x) or Successor(S,k):find smallest element in S that is larger than x .key(or key k).
- Predecessor(S,x) or Predecessor(S,k):find largest element in S that is smaller than x .key(or key k).

2.Operations

3.Treaps

3.1 Treap

A Treap (Binary-Search-Tree + Heap) is a binary tree in which each node has a key value ,and a priority value.

- The key values must satisfy the BST-property:
 - For each node y in left sub-tree of x : $y.key \leq x.key$
 - For each node y in right sub-tree of x : $y.key \geq x.key$
- The priority values must satisfy the MinHeap-property:
 - For each descendent y of x : $y.priority \geq x.priority$

3.2 Uniqueness of Treap

Given a set of n nodes with distinct key values and distinct priority values, a unique Treap is determined.

3.3 How to Build Treap

从空树堆开始，对于node x ，我们给它一个随机优先级，然后把它插入到Treap中。

Treap:a BST built with n insertions, in the order of increasing priorities.

3.4 Insert in Treap

- Step 1:Assign a random priority to the node to be added
- Step 2:Insert the node following BST-property
- Step 3:Fix MinHeap-property(without violating BST-property)

In Step 2 ,we use right-rotate and left-rotate,because it preserves BST property.

3.5 Remove in Treap

- Step 1:Use rotations to push-down the node till it is a leaf
- Step 2:Remove the leaf

3.6 Summary on Treap

Like a randomly built BST

Support ordered dictionary operations in $O(\log n)$ time,in expectation